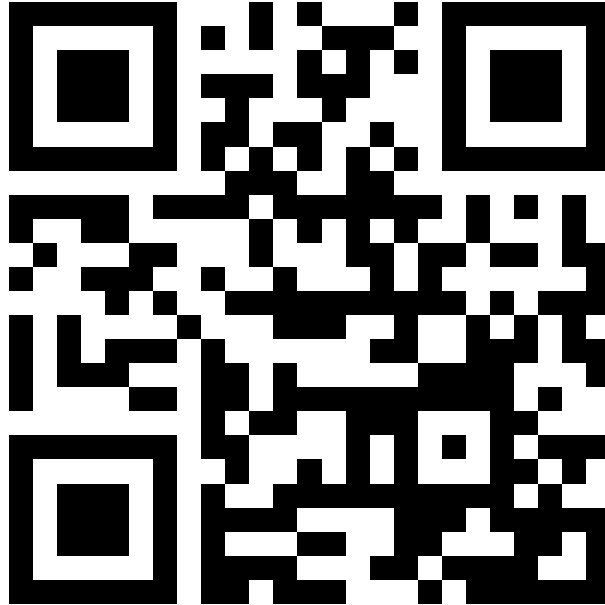


C++ Working Group



Contracts for C++ 26

Quick introduction

Contract assertions

```
1 float withdraw(int user_id, float amount)
2     pre(amount > 0.f)
3     post(r: r >= 0.f)
4 {
5     float current_balance = fetch_user_balance(user_id);
6     contract_assert(current_balance >= amount);
7     return set_balance(user_id, current_balance - amount);
8 }
```

Evaluation semantics

- ignore
- quick_enforce
- observe
- enforce

Contract Violation Handler

```
1 void handle_contract_violation(const std::contracts::contract_violation& v) {  
2     std::println("{} ", v.comment);  
3     std::println("{} ", std::stacktrace::current)  
4 }
```

Details

Not part of the proposal

- Virtual functions, pointers to functions
- Capture original parameter value
- `[[assume]]`-like optimization
- Set semantics in code
- Tag/group contract assertions
- Class invariants
- More...

Evaluation semantic is unspecified

- In theory
 - Each call to `pre/post/contract_assert` can use different semantic
- In practice
 - Controlled via flag
- The committee does not want to prescribe flags

Contract Assertions Are Transparent

- Not part of function signature, no mangling
- Do not affect ABI
- Not part of immediate context
- Cannot access properties via reflection or concepts

Predicates always parsed

```
1 #define NDEBUG
2
3 void f(int a) {
4     assert(a[0]);
5 }
```

```
1 // assume the ignore semantic
2 void f(int a) {
3     contract_assert(a[0]); // fails
4 }
```

What is the result?

```
1 // f.h
2 inline void f(int a) {
3     assert(a > 0);
4 }
5
6 // g.cpp
7 #define NDEBUG
8 #include <f.h>
9 void g() { f(1); }
10
11 // h.cpp
12 #include <f.h>
13 void h() { f(-1); }
```

Mixed mode – not ODR violation

```
1 // f.h
2 inline void f(int a) {
3     contract_assert(a > 0);
4 }
5
6 // g.cpp
7 // TU compiled with ignore
8 #include <f.h>
9 void g() { f(1); }
10
11 // h.cpp
12 // TU compiled with enforce
13 #include <f.h>
14 void h() { f(-1); }
```

SFINAE

Compiles

```
1 #include <contracts>
2
3 struct A {using foo = int; };
4 struct B {using bar = float; };
5
6 template<typename T>
7 void f(T, typename T::foo* = nullptr) pre(sizeof(typename T::foo) > 2) {}
8
9 template<typename T>
10 void f(T, typename T::bar* = nullptr) pre(sizeof(typename T::bar) > 2) {}
11
12 int main() {
13     f(A{});
14     f(B{});
15 }
```

Does not compile

```
1 #include <contracts>
2
3 struct A {using foo = int; };
4 struct B {using bar = float; };
5
6 template<typename T>
7 void f(T) pre(sizeof(typename T::foo) > 2) {}
8
9 template<typename T>
10 void f(T) pre(sizeof(typename T::bar) > 2) {}
11
12 int main() {
13     f(A{});
14     f(B{});
15 }
```

Where does my code live?

- **Where** – not specified
- **When** – specified
 - pre
 - After parameters are initialized
 - Before function body
 - post
 - After local variable destruction
 - contract_assert
 - Where control flow reaches them

Evaluation count is unspecified

```
1 int f(int a) pre(f1(a)) pre(f2(a)) post(r: f3(r));
```

- Order is specified
 - f2, f1 is not allowed
- Can repeat
 - f1, f2, f1, f1, f2, f3
- Can omit
 - f1, f3

Do not rely on side effects

```
1 int global = 0;
2
3 bool pred() {
4     std::println("Evaluate pred");
5     ++global;
6     return true;
7 }
8
9 void f() pre(pred()) {}
10
11 void g() {f();}
```

- global can be 0, 1, 42
- Side effects can be elided
 - “Evaluate pred” might be printed arbitrary amount of times

Constification

```
1 // Error
2 void f(int a) pre(++a > 0);
3
4 // OK
5 void f(int a) pre(++const_cast<int&>(a) > 0);
```

Constification

```
1 struct A{};
2
3 bool f(const A&) {
4     return true;
5 }
6
7 bool f(A&) {
8     return false;
9 }
10
11 void g(A a) pre(f(a)) {
12     std::println("{} ", f(a));
13 }
```

Constification

```
1 // Contract violation
2 void g(int a) pre(std::is_const_v<decltype(a)>) {}
3
4 // OK
5 void g(int a) pre(std::is_const_v<decltype((a))>) {}
```

Post requires const

- Function arguments referenced in post must be const
- Constness of arguments now matters!

```
1 // Error
2 int add_one(int a) post(r: r > a) {
3     return a + 1;
4 }
5
6 // Correct
7 int add_one(const int a) post(r: r > a) {
8     return a + 1;
9 }
```

If post didn't require const

```
1 float max(float a, float b) post(r: r >= a && r >= b) {  
2   a = b = 0;  
3   return 1;  
4 }  
5  
6 max(5, 6);
```

Exceptions

```
1 bool pred(int a) {  
2     if(a < 0) {  
3         throw std::runtime_error();  
4     }  
5     return true;  
6 }  
7  
8 void f(int a) pre(pred(a)){}  
9  
10 // OK  
11 f(1);  
12  
13 // Failed contract assertion  
14 f(-1);
```

Recursion

```
1 bool check(int a) {  
2     return a > 0;  
3 }  
4  
5 void f(int a) pre(check(a)) {}  
6  
7 void handle_contract_violation(const std::contracts::contract_violation&) {  
8     f(-1);  
9 }  
10  
11 f(-3);
```

Discussion

C++ Working Group

